

Chapter

03

LLM 래퍼 기술 동향과 구현 사례

김완석_ (주)넷비전텔레콤 기술고문
 오승일_ 한국식품연구원 선임연구원
 이성국_ (사)원정연구원 센터장
 송평중_ 전통연구협동조합 이사장
 전병천_ (주)넷비전텔레콤 대표

본 고는 LLM 실무 적용 시에 핵심인 래퍼 시스템의 보조 도구와 기술 동향을 살펴보고, 경량 LLM 래퍼 시스템 프로토타입 WALT를 제시한다. 래퍼는 입력 위생, 출력 요약과 정규화, 검색 증강과 도구 호출, 관측과 거버넌스, 비용 추적을 표준화된 오케스트레이션 계층으로 통합되며, 최근에는 벡터 데이터베이스, 정책 엔진, 오프라인·온라인 평가 하네스와의 결합으로 진화하고 있다. WALT는 생성형 AI와의 대화적·순환적 개발 방식인 바이브 코딩(vibe coding)으로 설계·구현·검증을 반복하며 구현하였고, 파이썬(Python) 언어와 플라스크(flask) 기반으로 PC 상에서 동작한다. 기능은 LLM 자동 선택, 로컬 캐시와 오프라인 모드, 이메일·전화 등 개인정보 마스킹, 서버 전송 이벤트 스트리밍을 포함하며, PC 등의 환경에서도 동작하도록 설계하여, 일반적 규격의 PC 환경에서도 생성형 AI 응용 시스템의 구현과 동작을 실험할 수 있게 하였다. 한편, 대규모 검색 증강, 장기 메모리, 정량 평가, 세분 권한과 거버넌스 정교화, 프롬프트 인젝션 대응 등의 구현은 향후 과제로 남아 있다.

I. 서론

생성형 AI에서 모든 도구·데이터·에이전트를 순서와 조건에 따라 묶어서 하나의 서비스로 제공하는 것을 체이닝(chaining)이라 하며, 이 운영 기능을 포괄적으로 관리하는 것을 오케스트레이션(orchestration)이라 한다. 이 운영계층을 LLM(Large Language Models)과 분리하여 흔히 래퍼(wrapper) 시스템 혹은 래퍼라고도 부르며, 챗GPT는 이 부분이 모델하

* 본 내용은 김완석 기술고문(☎ 042-931-4130, ws-kim@naver.com)에게 문의하시기 바랍니다.

** 본 내용은 필자의 주관적인 의견이며 IITP의 공식적인 입장이 아님을 밝힙니다.

***본 고는 2026년도 과학기술정보통신부 재원으로 한국식품연구원의 지원(E0220702)을 받아 수행된 연구성과입니다.

네스 · 맥락관리 · 안전성 가드레일 · 도구 오케스트레이션 · 검색증강 · 실행환경 · 응답생성기 · 기억 및 개인화 등으로 구성되어 있다.

LLM은 대규모 텍스트 데이터를 학습하여 의미와 맥락을 이해하여 사람처럼 문장을 생성하는 지능 모듈로서 요약 · 번역 · 질의응답 등의 자연어처리를 담당하는 핵심이고, 반면에 래퍼는 사전에 학습된 언어모델인 LLM과 더불어 서비스와 운영환경을 안정적이고 효율적으로 관리하는 프레임워크로 역할을 한다. 래퍼는 입출력 흐름을 관리하는 운영 계층인 동시에 입력 정규화, 개인정보(Personally Identifiable Information: PII) 마스킹, 모델 선택, 캐시 관리, 스트리밍 응답, 보안 인증, 로그 기록 등의 처리를 담당한다.

LLM 래퍼 응용을 개발하는 방법으로는 고수준 프레임워크 LangChain, LlamaIndex 등 또는 경량 서버형 Ollama, ChatBox 도구 등이 있다. LangChain은 GUI 도구를 보조적으로 제공하지만, 기본적으로 파이썬(Python) 코드로 lib를 제공하여 응용을 개발할 수 있다. LangChain은 이전보다 세밀하게 제어할 수 있게 개선되었으나 내부가 과도하게 추상화되어 여전히 세밀한 통제는 어렵다. Ollama도 Lib와 파이썬 언어로 구현하며 로컬 모델 실행이 쉽지만, 운영형 래퍼 기능 중 일부만 제공하기 때문에 구조 제어가 제한적이다[1]. 한편, 파이썬 언어로만 구현할 경우 개발비용과 유지보수에 대한 부담이 크지만, 원하는 대로 설계하거나 구조적 실험이나 최적화에는 유리하다.

이 개발 방법 중에서 파이썬 언어를 사용하여 LLM 운영 계층을 체계화하기 위한 경량 래퍼 프레임워크로 WALT(Wrapper for Adaptive LLM Tuning)를 정의하고, WALT는 LLM 자체가 아니라 LLM 호출 전후 과정을 관리하는 시스템 계층으로서의 프로토타입 구현을 시도하였다.

본 고는 다양한 형태의 래퍼 개발 지원 도구들로 LangChain, Haystack, LlamaIndex, Vertex AI 그리고 Ollama, ChatBox들을 통해 LLM 래퍼 시스템에 대한 기술 동향을 서술하고, 이어 WALT 프로토타입 시스템을 설계하고 구현한 결과와 추후 개선 방향을 중심으로 서술하고, 마지막으로 본 WALT 구현 사례에 대한 시사점을 정리한다.

II. LLM 래퍼 시스템 기술 동향

LLM은 자연어를 이해하여 문장을 생성하는 인공지능(Artificial Intelligence: AI) 기술로

대량의 데이터와 파라미터를 기반으로 학습되며, 입력된 문맥에 따라 확률적으로 다음 단어를 예측하는 방식으로 동작하는데, 이제 그 역할이 단순한 통계 예측을 넘어 복합적인 추론과 의미 구성 단계로 확장됨에 따라 단일 명령 수행을 넘어 도구 호출과 지식 검색을 통합한 대화형 에이전트 형태로 진화하고 있다. LLM의 기능은 언어 처리 그 자체이고, 이를 안정적이고 신뢰할 수 있는 서비스로 운영·관리하는 별도의 계층이 래퍼이며, 이 래퍼는 LLM을 서비스 운영 단위로 격상시키는 핵심 역할을 한다. 초기의 LLM 응용은 단순히 REST API로 LLM을 호출하였고, LLM 선택이나 자원 제어는 대부분 사용자의 수동 설정에 의존하였다. 이후에 LLM의 규모가 증가하고 기능의 다양화가 진행되면서 메모리 관리, 응답 지연, 비용 균형 등의 처리에 대한 문제들이 점차로 부각되었다. 이 문제들을 해결하기 위해 자원 자동 감지와 캐시 최적화, 품질·지연·비용의 동적 균형 등을 지원하는 래퍼가 등장하여 시스템 상황에 맞게 LLM을 고르고, 자주 쓰는 맥락을 저장해 두어 불필요한 계산을 줄이고, 긴 대화는 요약해 이어 붙이고, 개인정보는 보이지 않게 처리하고, 웹 서비스 환경에서는 CORS (Cross-Origin Resource Sharing) 정책을 적용한다. 이 오류는 JSON(JavaScript Object Notation) 형식으로 정리해 응답 처리하고 있는데, 이런 구성들이 래퍼가 갖추어야 할 기본 기능으로 굳어지고 있다.

[그림 1]은 LLM 래퍼의 일반적인 구조와 처리 흐름을 나타내고 있는데, 먼저 사용자의 입력이 인젝션 가드와 요약 전처리 단계를 거쳐 안전화 되고, 캐시 조회와 모델 선택 과정을 통해 생성기로 전달되면, 이어서 생성된 결과는 후처리와 로깅을 거쳐 사용자에게 반환된다. 이러한 구조를 통해 모델 간의 성능 편차와 자원 불균형 문제가 완화되어 제공하는 서비스가 안정화된다.

최근에는 세계 주요 기술 기업과 연구기관이 자체적 LLM 운영 래퍼 아키텍처를 제시하고, MCP(Model Context Protocol)는 원래 클로드를 만든 앤트로픽(Anthropic)이 2024년 말에 공개한 래퍼 층의 정보 교환을 위한 개방형 프로토콜이며, 2025년에 OpenAI · Microsoft도 쓰기 시작하면서 MCP가 모델



〈자료〉 ㈜넷비전텔레콤 작성

[그림 1] 일반적 LLM 래퍼 기본 구조와 데이터 흐름

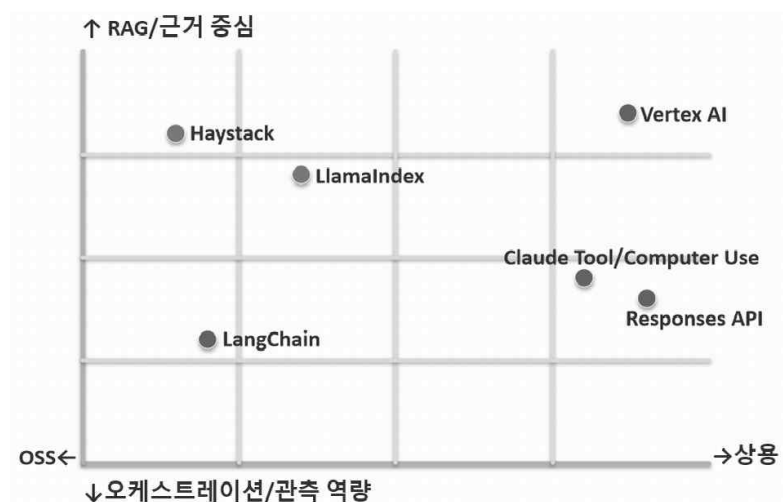
간 지식 교환 방식으로 정립되고, 또한 엔트로픽은 Constitutional AI를 활용하여 스스로 응답을 비평·수정하는 정책 기반 응답 통제를 시도하고 있다. 메타(Meta)의 LLaMA Guard는 안전성 검증 및 필터링 기능을 강화하고 있으며, 구글(Google)의 Vertex AI는 Gemini API와 더불어 멀티모달 입력을 통합하는 래퍼 계층을 제공하고 있다. 국내에서는 금융, 법률, 공공 부문에서 보안 특화형의 래퍼가 빠르게 확산되고 있는데, 특히 솔트룩스, 업스테이지, 네이버 클로바 등이 도메인 특화형 래퍼를 개발하고 있다[2].

한편, LangChain, LlamaIndex, Haystack 등의 오픈소스 프레임워크는 체이닝, 커넥터, 검색 및 임베딩 그리고 외부 도구 연계를 모듈화한 라이브러리로써 오케스트레이션에 대한 구성을 가속하고 있으며, 이들은 LLM 호출에 앞서 데이터 흐름을 구성하고, 문서 인덱싱 및 검색 파이프라인 구축을 지원하는 소프트웨어 도구 군이다. 반면, 상용 플랫폼인 OpenAI Responses API, Anthropic Claude Tool/Computer Use, Google Vertex AI 등은 도구 호출, 정책 적용, 관측 및 모니터링 기능을 통합한 관리형 오케스트레이션 환경으로 플랫폼 수준의 래퍼 계층을 구성하고 있다.

이들 오픈소스 라이브러리(예; LangChain, LlamaIndex)와 상용 플랫폼(예; Vertex AI, Claude Tool)들은 단순 API 게이트웨이에서 출발하여 LLM의 학습, 추론, 운영, 평가 등에 대한 모든 단계를 통합 관리함에 따라, 결과적으로 래퍼는 모델 호출 기능뿐만 아니라 보안·거버넌스·실행 제어·추적 및 모니터링을 일체화한 운영 계층으로 진화하고 있으며, 사실상 표준적 참조모델로 구체화되고 있다.

Ollama, ChatBox 등도 로컬 모델 실행 환경에서 주목받고 있는데, Ollama는 모델 다운로드, 관리 및 로딩을 단일 명령 기반으로 단순화하여 로컬 LLM 실행·배포하는 런타임 계층을 제공한다. 이는 개인 및 소규모 환경에서 모델 구동과 관리의 편의성이 높지만, 보안·관측·도구 호출 등의 운영형 래퍼의 전 범위를 포함하지는 않아서 오케스트레이션 및 관측 기능 전체를 제공하는 래퍼라기보다는 로컬 LLM 실행기 역할에 가까우며 로컬 환경용 프롬프트 운영 및 인터페이스 제공 도구로 분류되는 것이 타당하다.

[그림 2]의 래퍼 발전 방향을 ‘오케스트레이션 및 관측역량(가로축)’과 ‘RAG(Retrieval-Augmented Generation) 및 근거중심(세로축)’으로 나누어서 비교하여 보면, Haystack과 LlamaIndex는 RAG 축의 기능을 중심으로 지원하고, LangChain은 오케스트레이션 축의 운영 및 관리를 중심으로 지원하고 있으며, Vertex AI는 두 축을 동시에 지원하는 기능



〈자료〉 ㈜넷비전텔레콤 작성

[그림 2] LLM 래퍼 시스템 프레임워크 비교

및 관리형 스택으로 평가되고, Responses API와 Claude Tool/Computer Use는 도구 호출과 정책통합 면에서 높은 성숙도를 보인다. 이러한 래퍼의 진화 방향은 ‘오케스트레이션 +RAG+관측 역량 · 정책통합’ 구조로 수렴하고 있으며, WALT는 그중에서 “경량 환경에서도 필수 운영 기능에 국한한 실험적 래퍼”라는 점이 특징이다.

III. WALT 프로토타입 구현 사례

WALT는 기능적으로는 LLM API 호출과 더불어 LLM 운영관리, 로그 추적, 성능 관찰 등의 기능들을 포함하면서, 하드웨어적 환경으로는 GPU가 없거나 메모리가 부족한 16~32GB급 PC에서도 구동이 가능한 운영 프레임워크를 실험적으로 설계 · 구현하였다. 즉, WALT의 구현 의도는 LLM 자동 선택과 운영자의 개입을 최소화하면서 보안 및 로그 관리 체계를 내장하여 로컬 LLM 기반 응용을 실험적으로 개발하려는 개인이나 소규모 환경을 주요 동작 대상으로 하였기 때문에, 이러한 구현 의도에 따라 래퍼를 구성하는 다양한 기능 중에서 불필요한 기능들은 제거하였고, 남은 구조의 래퍼가 최소한의 본질적 래퍼의 역할을 할 수 있도록 고려하였다.

한편, WALT의 프로그램 코드를 바이브 코딩(vibe coding)으로 생성하는 과정은 설계규격

도출, 코드 초안 생성, 수정작업으로 나누어 순차적으로 진행하였다. 먼저, 설계규격 도출 작업은 인터넷 자료와 생성형 AI를 사용하여 반복적으로 개선해 가면서 설계규격 완성도를 높인다. 최초의 WALT 설계규격은 시스템이 자동으로 학습 데이터를 생성·정제하고, 지속적 재학습을 통해 모델 지능을 진화시키며, 사용자 피드백과 평가 체계를 통해 자기 개선하는 통합 시스템으로 설계하였으나, 구현 시에는 설계 내용과는 다르게 기존 LLM 모델을 로드하여 사용하는 현실적인 방법으로 설계를 수정하였다. 실제로 코드 초안의 생성은 완성된 설계규격을 지시문으로 하여 생성형 AI를 통해 설계규격에 따른 코드 초안을 생성하였다. 그리고 코드 초안 수정작업은 직접 PC 환경에서 실행시켜 오류를 확인하고, 오류 내용에 대한 수정 지시문을 생성형 AI의 프롬프트에 입력시켜서 코드를 수정하였고, 오류가 수정된 코드를 PC 환경에서 다시 실행하여 오류를 재확인하고, 추가 발생한 오류 내용에 대한 수정 지시문을 다시 생성형 AI의 프롬프트에 입력시켜서 코드를 다시 개선하는 바이트 코딩 과정을 N번 반복하여 코드를 생성하였다.

WALT 시스템은 모델 및 토큰라이저 로딩은 로컬 캐시 우선, 원격 폴백 보조 구조로 운영되며, 16~32GB 메모리 환경을 기준으로 16~24GB 구간에서는 6B 급 모델을, 24~32GB

```

명령 프롬프트 - python walt03f.py gemma2:9b-instruct-q4_K_M
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

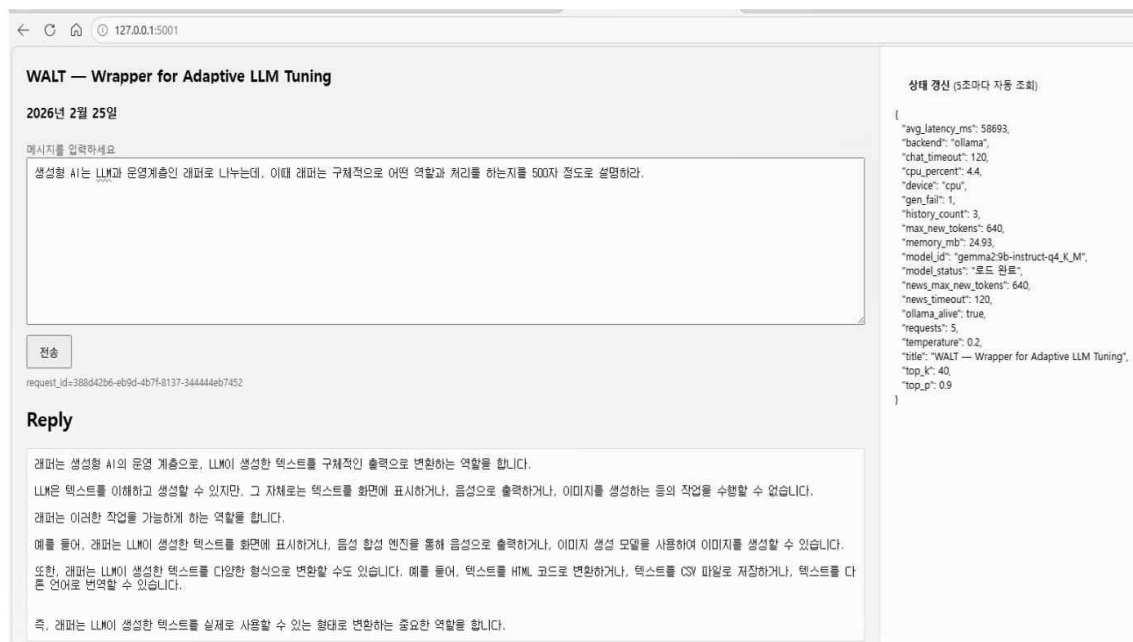
C:\Users\Wnetvision>python walt03f.py gemma2:9b-instruct-q4_K_M
2026-03-03 10:25:57,035 [INFO] Starting WALT server...
2026-03-03 10:25:57,036 [INFO] Using backend=ollama, model_id=gemma2:9b-instruct-q4_K_M, ollama_base_url=http://1:11434
2026-03-03 10:25:57,039 [INFO] 모델 로딩/서비스 동작 중...
* Serving Flask app 'walt03f'
* Debug mode: off
2026-03-03 10:25:57,078 [INFO] * WARNING: This is a development server. Do not use it in a production de
Use a production WSGI server instead.*
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.3.57:5001
2026-03-03 10:25:57,079 [INFO] * Press CTRL+C to quit*
2026-03-03 10:26:14,379 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:14] "GET / HTTP/1.1" 200 -
2026-03-03 10:26:14,607 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:14] "GET /status HTTP/1.1" 200 -
2026-03-03 10:26:14,655 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:14] "[33mGET /favicon.ico HTTP/1.1+[0m" 404
2026-03-03 10:26:19,560 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:19] "GET /status HTTP/1.1" 200 -
2026-03-03 10:26:24,564 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:24] "GET /status HTTP/1.1" 200 -
2026-03-03 10:26:27,042 [INFO] 모델 로딩/서비스 동작 중...
2026-03-03 10:26:29,554 [INFO] 192.168.3.1 - - [03/Mar/2026 10:26:29] "GET /status HTTP/1.1" 200 -
  
```

〈자료〉 ㈜넷비전텔레콤 작성

〔그림 3〕 WALT 프로토타입 부팅과 서비스 과정

구간에서는 7B 급 모델을 자동으로 로드되게 하여 GPU가 없는 환경에서도 매우 느리지만 CPU 기반 실행이 가능하도록 설계하였다.

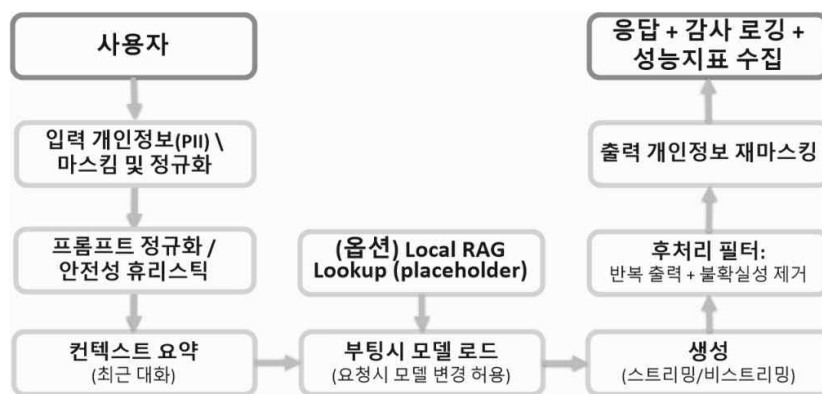
[그림 3]은 바이브 코딩으로 데이터 파이프라인 구조로 코딩한 WALT 프로토타입 부팅 과정과 웹서버 설정 및 LLM 로딩 그리고 서비스 처리 모습을 보여 주고 있으며, [그림 4]는 WALT 프로토타입의 사용자 인터페이스 화면으로 메시지 입력 창과 메시지 서버 전송 버튼 그리고 LLM이 생성한 결과를 출력하는 응답창이 동작하는 모습과 현 동작 시스템에 대한 실시간 정보를 표시하고 있으며, 이 프로그램 코드는 입력 전처리, 요약 및 캐시, 모델 선택, 응답 생성 및 후처리, 보안 및 감사와 같이 5개의 계층으로 나누어져 있다. 이들 각 계층의 역할을 차례대로 살펴보면, 입력 전처리 계층은 사용자 질의 정규화, 프롬프트 인젝션(prompt injection) 위험 감시, 개인정보 마스킹 등의 입력 내용에 대한 전처리들을 수행한다. 이어 요약 및 캐시 계층은 BART(Base Auto-Regressive Transformer) 기반 요약 수행과 LRU(Least Recently Used) 캐시를 통해 최근의 이력 대화의 맥락을 재활용하여 입력에 대한 길이의 제한이 있는 환경에서도 문맥을 유지한다. 모델 선택 계층은 부팅 시에 응용 프로그램 사용이 가능한 메모리 양을 측정하여 동작 중에 메모리 오류를 유발하지 않을 수준



〈자료〉 ㈜넷비전텔레콤 작성

[그림 4] WALT 프로토타입의 사용자 인터페이스 화면 모습

크기의 LLM으로 TinyLlama(1.1B), Phi-2, GPT4All-J, Mistral-7B 등으로 선택하여 로드한다. 그리고 응답 생성 및 후처리 계층은 선택된 LLM에서 생성된 응답에서 불필요한 반복이나 내용을 제거하는 동시에 JSON 형식으로 결과를 구조화하며, 필요 시에는 SSE(Server-Sent Events) 기반 스트리밍으로 긴 응답을 단계적으로 전송할 수도 있다. 보안 및 감사 계층은 Bearer 토큰 인증, CORS 정책, 상관관계 ID(Correlation ID) 기반 감사 로깅, 상태 점검 엔드포인트(/status, /metrics, /debug_log)를 통해 서비스 접근과 운영 흐름을 통제하고, 시스템 활동을 추적한다. [그림 5]에서 WALT의 구조와 데이터 흐름을 살펴보면, 입력 정규화와 인젝션 가드, 요약 및 캐시 관리, 자동 모델 선택, 후처리, 감사 로깅의 처리 과정을 단계적으로 처리하고 있다. 한편, WALT의 부팅 과정의 역할을 살펴보면 시스템의 가용 메모리를 기동 시에 측정하고 이를 기준으로 하여 적절한 LLM을 자동 선택하는데, 이 구조는 장시간 실행 시 로그 폭증으로 인한 저장소 오류를 예방하고, 처리에 대한 시간의 지연을 줄여 준다. 이어서 입력 전처리와 응답 생성, 후처리 과정의 역할을 살펴보면 입력 필터링은 인젝션 공격과 민감정보 노출을 효과적으로 차단하는 역할을 하며, 응답 처리 시에 LRU 캐시의 히트율이 높을수록 응답 속도는 향상되며, 요약의 정확도는 캐시된 문맥의 품질에 영향을 미친다. 플라스크(Flask) 서버의 역할을 살펴보면, 주요 엔드포인트는 /chat, /status, /metrics, /debug_log의 네 가지로 구분하여 구성하였다. /chat은 대화 요청을 처리하고, /status는 모델 로딩 상태와 자원 정보를 확인하며, /metrics는 요청 빈도와 응답지연을 집계하고, /debug_log는 운영 로그를 조회하는 기능을 제공하는 역할을 한다. 한편, CLI



〈자료〉 ㈜넷비전텔레콤 작성

[그림 5] WALT의 구조와 데이터 흐름

(Command Line Interface) 도구인 waltcli.cmd는 서버의 초기화, 실행, 중단, 모델 선택, 로그 조회 등에 대한 명령을 콘솔 환경에서 사용할 수 있게 제공한다.

IV. WALT 프로토타입 검증

본 검증은 코드 수준의 스모크 테스트(smoke test) 형태로 진행하였으며, 검증의 목적은 모델 로딩과 응답 생성의 가능성 그리고 보호·관측·캐시 기능이 의도한 대로 작동하는지를 확인하는 데 있었다. 시험에 사용된 PC의 물리 메모리는 16GB였으나 실제 가용 메모리는 15GB 미만이었으며, 부팅 과정에서 메모리 측정값에 따라 자동으로 TinyLlama 1.1B 모델이 선택되어 로딩되는 과정을 확인했으며, 이 조건에서 6B 및 7B 급 모델의 로딩도 시도했으나 로딩은 중단 또는 할트(halt) 현상이 발생하였다.

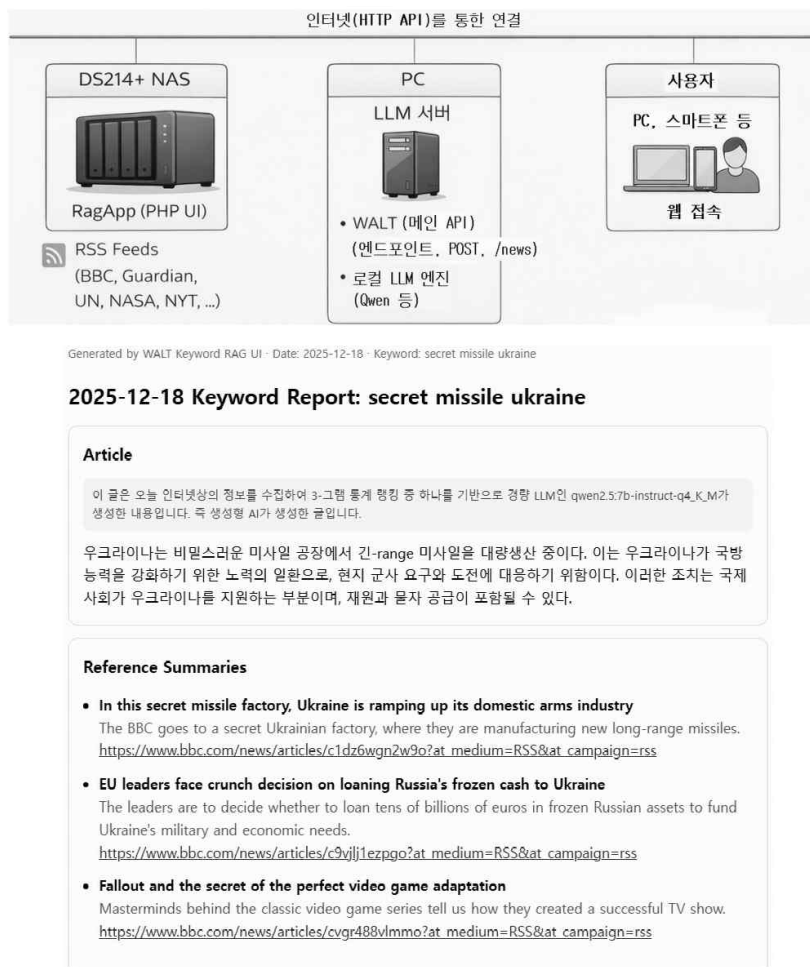
전화번호나 이메일 등의 개인정보는 정규식 기반의 PII 마스킹이 적용되었고, “지시 무시”와 같은 프롬프트 인젝션류 입력은 자동으로 차단되었으며, 모든 요청에는 상관관계 ID와 토큰 인증 체계가 적용되어 감사 로그에 기록되었으며, 보안 및 거버넌스 기능은 구현된 코드의 내용에 따라 정상적으로 작동하였다[4]. 임의의 질의를 여러 차례 반복하여 시도를 해보면 질의가 누적됨에 따라 응답 지연 시간이 점진적으로 완화되는 듯이 체감되었는데, 이러한 현상은 동일한 입력 조건 하에서 맥락 재활용 및 캐시 메커니즘이 부분적으로 활성화된 결과로 추정되지만, 이는 어디까지나 사용자가 체감한 수준의 경험적 관찰에 근거한 것으로 표준화된 마이크로벤치마크나 부하시험을 체계적 절차와 조건에 따라 요약 기반 맥락 압축 기능을 확인한 것은 아니다. FAISS(Facebook AI Similarity Search) 등의 외부 벡터 인덱스나 메모리상의 벡터 데이터를 검색하는 RAG 기능 자체는 구현하지 않았으나, 외부 지식 검색 결과를 삽입할 수 있는 훅(hook) 형태의 인터페이스와 프롬프트 삽입 지점은 준비하여 두었다[3].

시스템 상태 점검을 위한 /health, /status, /metrics, /debug_log 등의 엔드포인트는 구현되어 있으나, 현재는 모델 로딩 상태, 요청 처리 현황, CPU/메모리 사용량, 내부 로그 및 디버그 메시지 등의 기초적 시스템 정보만 수집되는 상태이다. 한편, 응답이 지연되는 요청을 자동으로 차단하는 타임아웃 가드를 적용하여 시스템 장애 발생 혹은 장애 확산을 방지하게 되어 있으며, 실시간 응답을 위한 선택적 스트리밍(/chat?stream=true) 응답도

정상적으로 동작함에 따라, 현재 시스템은 기본적인 운영 상태 점검 및 안정성 보호 메커니즘은 구현된 상태이다.

요약하면, WALT는 입력 위생, 자동 모델 선택, 캐시·요약, 기본 보안·관측 기능이 정상적으로 작동하는 경량 래퍼 아키텍처로 확인되었다. 다만, 메모리 제약 환경에서는 6B·7B 모델 로딩이 불안정하며, RAG·고급 관측·정량 평가 등은 미완성 단계에 머물러 있다. 전체적으로 “경량 환경에서 작동이 가능한 래퍼”임을 확인한 수준이다.

그리고 TinyLlama 1.1B 모델 대신에 Qwen 2.5 7B LLM으로도 돌렸는데 역시 느리기는 하지만 답변은 잘하는데 가끔 중국어가 나왔다. 아마 Qwen은 중국 알리바바 클라우드가



〈자료〉 ㈜넷비전텔레콤 작성

[그림 6] WALT 상의 RAG 응용 서비스 구현 예시

만들어서 나오는 결과로 보인다. TinyLlama 1.1B 모델에 비하면 Qwen 2.5 7B LLM은 사용상에 무리가 없는 성능을 보여 주었으며, 7B는 파라미터가 70억 개라는 뜻인데 GPU 없이는 OOM이나 할트로 돌지 않지만, 사용한 Qwen 2.5 7B의 데이터셋은 4비트 양자화 처리된 것으로 실제 크기보다 1/4 이하 크기인 4.7GB 정도이다. 한편, [그림 6]과 같이 Qwen 2.5 7B LLM과 php 프로그램 언어로 약 1,000라인의 RAG 응용 소프트웨어를 바이트 코딩으로 구현하여 NAS 장치에서 RAG 응용을 동작시키고 PC 상의 LLM 서버가 인터넷 신문 기사를 스스로 생성하도록 하였다. 즉, NAS 장치의 RAG 응용이 인터넷상에서 오늘 데이터를 수집하고, 수집된 자료 중의 상위 출현 빈도수의 키워드와 수집 정보를 요약하여 PC 측에 보내면, PC 상의 Qwen 2.5 7B LLM 백엔드 서버가 기사를 생성하여 NAS 측 RAG 응용에 다시 되돌려 준다. 이 데모는 주어진 키워드로 5~6 문장을 생성하는데 특별한 엔드포인트로서 /news를 WALT 상에 준비하여 사용했는데, 기존 처리시간이 60초인 /chat보다 많은 처리시간 120초를 할당하였다. 기사 1건을 생성하는데 시간이 다소 많이 소요되는 것과 가끔 한글 기사 중에 중국어가 혼재되는 것을 제외하면 특별한 문제 없이 잘 동작하였다.

V. 향후 과제

WALT 시스템의 주요 개선 사항은 검색 증강 생성(RAG) 기능의 정밀화를 통해 각 응답이 “LLM만의 결과”가 아닌 정책 기반의 소스 우선순위를 적용하고 “검증 가능한 근거 경로”를 제공하는 체계를 제공하여 환각(hallucination)을 최소화할 수 있도록 한다. 따라서 현재의 TF-IDF 기반 검색이나 소규모 FAISS 인덱스 수준을 넘어, 벡터 데이터베이스(Vector DB)에 기반한 정확한 근거 검색과 문단·문서 단위 출처 제시가 가능해야 한다. 그리고, 미니 배치 학습 수준의 현재의 도메인 적응력 한계를 보완하기 위해 LoRA(Low-Rank Adapter)와 어댑터 기반 미세 조정과 RLHF(Reinforcement Learning with Human Feedback)을 결합하여, 메모리 제약 환경에서도 효율적 파인튜닝이 가능하도록 지원해야 한다[3].

최근 LLM 서비스가 텍스트 중심 구조를 넘어 음성·이미지·영상·센서 등 실환경 데이터로 확장되고 있어, WALT도 이에 대응할 수 있는 멀티모달 처리 체계를 단계적으로 확보해야 한다. 이러한 변화는 단순한 입력 형식의 다양화를 넘어 WALT 구조 전반의 확장을 요구하므로, 향후 버전에서 멀티모달 기반의 운용 방안을 지속적으로 검토할 필요가 있다.

[표 1] WALT 개선 필요 대상

분야	개선 항목	필요성
RAG/근거 기반 생성	벡터DB 연계, 출처 제시, 근거 경로 정밀화	환각 최소화·정확성 확보
모델 적응력	LoRA/Adapter, RLHF 기반 경량 파인튜닝	도메인 적합성 향상
멀티모달 확장	음성·이미지·센서 입력 지원	실제 산업 데이터 대응
운영·관측·거버넌스	정책 기반 제어, 고급 로그·추적, 안전성 강화	실서비스 수준 운영 요건
장기 메모리·캐시 고도화	대화 지속성 강화, 비용 절감	사용자 경험 및 효율 향상

〈자료〉 ㈜넷비전텔레콤 작성

또한, WALT의 경량적 특성을 유지하는 범위 안에서 Whisper나 CLIP 혹은 Gemini 등의 외부 API를 보조 처리 통로로 활용하는 방안도 검토되어야 한다. 그리고 보안·거버넌스 영역에서는 운영 과정의 신뢰성과 규제 대응 기능을 지속적으로 유지할 수 있는 구조적 기반 마련도 요구된다.

[표 1]의 내용과 같이 WALT의 향후 발전 축은 검색 증강과 근거 기반 생성의 정밀화, 재학습 비용을 낮추는 분산 학습 체계의 확보, 멀티모달 입력의 단계적 확장, 그리고 보안·거버넌스 체계의 고도화로 집약되며, 이는 경량 환경에서 신뢰할 수 있는 LLM 래퍼를 구현하기 위한 기술적 로드맵이라고 할 수 있다.

VI. 결론

본고는 LLM 래퍼 시스템의 구조와 기능 그리고 보조 도구와 기술 동향을 개관하고, 실제 서비스 환경에서 프로토타입 WALT가 PC 수준의 일반적 실행 환경에서 경량 LLM을 운용하기 위한 래퍼 계층이 수행해야 할 핵심 기능의 동작과 한계를 확인하였다.

LLM 서비스의 경쟁력은 단일 모델의 성능에만 좌우되지 않고, 이를 감싸는 래퍼 계층의 안정성·신뢰성·규제 대응력에도 의존한다는 점을 고려할 때, WALT가 경량 환경에서도 자동 모델 선택, 캐시 기반 맥락 관리, 입력 보안, 타임아웃 스트리밍, 운영 로그 및 지표 관리와 같은 기능들의 구현이 가능함을 보여 주었다.

또한, WALT는 LangChain이나 Ollama와 같은 도구를 사용하지 않고 파이썬 언어로 구현함으로써, 시스템 RAM을 확인하여 적합한 LLM 모델을 선택한다던가, 로드(load) 실패 시에 fallback을 한다던가, 사용자 질문이 쌓여도 최근 대화 8개를 추출하여 자동으로 요약

하고 히스토리를 압축하는 등의 래퍼 개발 도구들이 제공하지 못하는 부분들도 구현할 수 있었다.

정책적 측면에서는 경량 LLM 래퍼 기술 개발을 지원하기 위한 공공 연구 투자와 대학·연구기관·산업계 간 협력체계를 통한 기술개발·인력양성이 병행되어야 하며, 프레임워크 및 모델 간 상호 운용 표준화 노력까지 더해져야 한다.

이러한 기술적·정책적 기반이 조성될 때 “경량 환경에서 신뢰할 수 있는 LLM 래퍼”는 구체화하고 개념 검증단계에서 실제 서비스 운영 구조로 확장될 수 있다.

● 참고문헌

- [1] Park, J., Kim, H. & Lee, S., Lightweight Deployment of LLM Wrappers for On-Premise Environments, Journal of KIISE, 51(2), 2024, 123-138.
- [2] Cho, Y., Han, J. & Kang, M., Governance and Security in Wrapper Systems for Generative AI, Electronics and Telecommunications Research Journal, 39(3), 2024, 233-249.
- [3] Hugging Face., Transformers: State-of-the-Art Natural Language Processing, GitHub Repository, 2024.
- [4] Ministry of Science and ICT(MSIT), AI Security Guidelines for Generative Models, MSIT White Paper, 2023.